Chapter 1 Unsupervised Learning

Introduction

- *Dimensionality reduction*
- *Cluster*

All methods we have discussed so far fall under the category of **supervised learning**, where the training process relies on labeled data and the model is explicitly guided by pairs (x, y). In contrast, **unsupervised learning** operates without labels: the goal is to learn the underlying structure of the data and to approximate the distribution of datapoints x itself.

Definition 1.1 (Unsupervised Learning)

Let $\mathcal{X}$ be an unlabeled dataset. Unsupervised learning aims to learn a mapping $f : \mathcal{X} \to \mathcal{Z}$ or a generative model $p_\theta(x)$ that captures the intrinsic structure of the underlying data distribution $p(x)$. Typical objectives include clustering, dimensionality reduction, density estimation, and representation learning. Importantly, no labeled pairs (x, y) are observed during training. ♣

Unsupervised learning can be divided into several tasks:

1. Dimensionality reduction: datapoints are often lie in high-dimensional manifold, while the dimension that really matter only contribute as a small part. Dimensionality reduction is the process to train the model to find these crutial dimension from enormous amount of dimensions.
2. Clustering. Train model to group similar data togather.
3. Generate models. Various generative model, like SoRA, LLM/GPT, are trained to learn how to generate contents like text or image by learning the distribution of the target context.

1.1 Dimensionality reduction

1.1.1 Principal Component Analysis (PCA)

Theorem 1.1 (Principle of PCA)

PCA seeks a direction w (with $\|w\| = 1$) along which the projected data $w^\top x$ achieves the maximum possible variance. *Formally, the goal is to maximize*

$$\mathrm{Var}(w^\top X). \tag{1.1}$$

♡

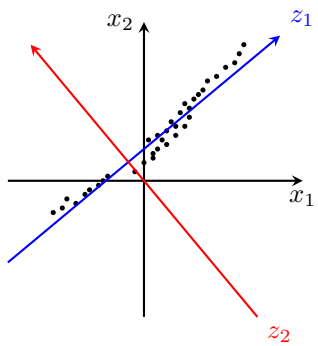
Let the dataset be

$$X = \begin{pmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_n^\top \end{pmatrix} \in \mathbb{R}^{n \times d}, \tag{1.2}$$

with sample mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i. \tag{1.3}$$

Each $x_i \in \mathbb{R}^d$ is an observation.



The empirical covariance matrix is defined as

$$\Sigma = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(x_i - \bar{x})^\top \in \mathbb{R}^{d\times d}. \tag{1.4}$$

Entrywise,

$$\Sigma_{jk} = \frac{1}{n}\sum_{i=1}^{n}(x_i^{(j)} - \bar{x}^{(j)})(x_i^{(k)} - \bar{x}^{(k)}). \tag{1.5}$$

Remark

The covariance matrix defined with the factor $\frac{1}{n}$ is technically a biased estimator of the true population covariance. The unbiased version uses $\frac{1}{n-1}$ instead. However, when n is large, the difference between $\frac{1}{n}$ and $\frac{1}{n-1}$ is negligible:

$$\frac{1}{n} = \frac{1}{n-1}\left(1 - \frac{1}{n}\right), \tag{1.6}$$

and thus affects the covariance only by a vanishing scalar factor.
Moreover, PCA depends solely on the eigenvectors *of the covariance matrix, i.e., the principal directions. Multiplying Σ by any positive constant does not change its eigenvectors. Therefore, for PCA, the choice between $\frac{1}{n}$ and $\frac{1}{n-1}$ has no effect on the principal components.*

Lemma 1.1 (Variance of a Linear Projection)

Let w be any unit vector in $\mathbb{R}^d$. The projection of a data point x_i onto w is the scalar

$$z_i = w^\top x_i. \tag{1.7}$$

The sample mean of the projected data is

$$\bar{z} = \frac{1}{n}\sum_{i=1}^{n} z_i = \frac{1}{n}\sum_{i=1}^{n} w^\top x_i = w^\top \bar{x}. \tag{1.8}$$

Thus

$$z_i - \bar{z} = w^\top(x_i - \bar{x}). \tag{1.9}$$

The sample variance of the projected data is

$$\mathrm{Var}(w^\top X) = \frac{1}{n}\sum_{i=1}^{n}(z_i - \bar{z})^2 = \frac{1}{n}\sum_{i=1}^{n}\left(w^\top(x_i - \bar{x})\right)^2. \tag{1.10}$$

Using the identity

$$(w^\top a)^2 = w^\top(aa^\top)w,$$

we obtain

$$\mathrm{Var}(w^\top X) = w^\top\left(\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(x_i - \bar{x})^\top\right)w = w^\top \Sigma w. \tag{1.11}$$

♡

Let u_1 be the unit projection vector on $\mathbb{R}^d$ to project each x_i onto a 1-D space, then PCA is equivalent to solve the below problem

$$\underset{\|u_1\|=1}{\operatorname{argmax}} \; u_1^\top \Sigma u_1. \tag{1.12}$$

By introducing a Lagrange multiplier λ for the constraint $u_1^\top u_1 = 1$, we consider the Lagrangian

$$\mathcal{L}(w, \lambda) = u_1^\top \Sigma u_1 - \lambda\big(u_1^\top u_1 - 1\big). \tag{1.13}$$

Taking the derivative with respect to u_1 and setting it to zero gives

$$\frac{\partial \mathcal{L}}{\partial u_1} = 2\Sigma u_1 - 2\lambda u_1 = 0, \tag{1.14}$$

hence

$$\Sigma u_1 = \lambda u_1. \tag{1.15}$$

The derivative with respect to λ enforces the constraint

$$\frac{\partial \mathcal{L}}{\partial \lambda} = -\big(u_1^\top u_1 - 1\big) = 0 \quad \Longrightarrow \quad u_1^\top u_1 = 1. \tag{1.16}$$

Therefore, any maximizer u_1 **must be an eigenvector of** Σ, with λ equal to the corresponding eigenvalue. Since

$$u_1^\top \Sigma u_1 = \lambda \tag{1.17}$$

for any unit eigenvector u_1, the optimization problem is solved by choosing u_1 as the eigenvector associated with the **largest eigenvalue of** Σ. This u_1 is the first principal component.

After obtaining the first principal component u_1, the second principal component u_2 is defined as the direction that maximizes the projected variance subject to two constraints:

$$\max_{u_2} \; u_2^\top \Sigma u_2 \tag{1.18}$$

subject to

$$u_2^\top u_2 = 1, \qquad u_2^\top u_1 = 0. \tag{1.19}$$

The first condition enforces unit length; the second ensures orthogonality to u_1.

Introduce Lagrange multipliers λ_2 and α. Consider the Lagrangian

$$\mathcal{L}(u_2, \lambda_2, \alpha) = u_2^\top \Sigma u_2 + \lambda_2(1 - u_2^\top u_2) + \alpha\, u_2^\top u_1. \tag{1.20}$$

Taking the derivative with respect to u_2 and setting it to zero yields

$$\frac{\partial \mathcal{L}}{\partial u_2} = 2\Sigma u_2 - 2\lambda_2 u_2 + \alpha u_1 = 0. \tag{1.21}$$

Rearranging the stationarity condition gives

$$\Sigma u_2 = \lambda_2 u_2 - \frac{\alpha}{2} u_1. \tag{1.22}$$

Using the orthogonality constraint $u_1^\top u_2 = 0$ and the fact that

$$\Sigma u_1 = \lambda_1 u_1, \tag{1.23}$$

we obtain

$$u_1^\top \Sigma u_2 = \lambda_2\, u_1^\top u_2 - \frac{\alpha}{2} u_1^\top u_1 = -\frac{\alpha}{2}. \tag{1.24}$$

On the other hand,

$$u_1^\top \Sigma u_2 = (\Sigma u_1)^\top u_2 = \lambda_1 u_1^\top u_2 = 0. \tag{1.25}$$

Thus $\alpha = 0$, and the stationarity condition reduces to the eigenvalue equation

$$\Sigma u_2 = \lambda_2 u_2. \tag{1.26}$$

Therefore u_2 must be an eigenvector of Σ associated with the second largest eigenvalue.

By induction:

Theorem 1.2

The first K principal components are exactly the eigenvectors of the covariance matrix Σ corresponding to its K largest eigenvalues.

♡

Denote these eigenvectors by

$$U_{1:K} = (u_1, u_2, \cdots, u_K) \in \mathbb{R}^{d\times K}. \tag{1.27}$$

Each u_i satisfies$\Sigma u_i = \lambda_i u_i$, $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_K$.

Given the matrix of the top K principal directions $U_{1:K}$, the projection of a data point $x \in \mathbb{R}^d$ onto the K-dimensional PCA subspace is $z = U_{1:K}^\top x \in \mathbb{R}^{n\times K}$. For a dataset $X \in \mathbb{R}^{n\times d}$, the projected data matrix is

$$XU_{1:K} \in \mathbb{R}^{n\times K}. \tag{1.28}$$

This reduces the dimensionality from d to K while retaining the directions of maximal variance.

Definition 1.2 (Centered Data Matrix)

Let the centered data matrix be

$$\hat{X} = \begin{pmatrix} x_1^\top - \bar{x}^\top \\ x_2^\top - \bar{x}^\top \\ \vdots \\ x_n^\top - \bar{x}^\top \end{pmatrix} \in \mathbb{R}^{n\times d}. \tag{1.29}$$

Each row of $\hat{X}$ is a mean-subtracted data point.

♣

The empirical covariance matrix can be written compactly as

$$\Sigma = \frac{1}{n}\hat{X}^\top \hat{X}. \tag{1.30}$$

Theorem 1.3 (Eigen-Decomposition of the Covariance)

Because Σ is real symmetric, it admits an orthogonal eigendecomposition:

$$\Sigma = U\Lambda U^\top, \tag{1.31}$$

where U is an orthogonal matrix whose columns are eigenvectors and Λ is a diagonal matrix containing the eigenvalues of Σ.

♡

In practice, numerical libraries such as `numpy.linalg.eig` or `numpy.linalg.eigh` compute the eigen-decomposition of Σ directly. The eigenvectors $\{u_1, \ldots, u_d\}$ form the principal directions of the data.

1.1.2 Singular Value Decomposition (SVD)

Consider the singular value decomposition of the centered data matrix:

$$\hat{X} = U_X S V^\top. \tag{1.32}$$

$$\hat{X} \in \mathbb{R}^{n\times d} = \underbrace{U_X}_{\mathbb{R}^{n\times r}} \underbrace{\begin{pmatrix} \sigma_1 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & \cdots & 0 \\ \vdots & & \ddots & & & \vdots \\ 0 & \cdots & 0 & \sigma_n & \cdots & 0 \end{pmatrix}}_{\mathbb{R}^{r\times r}} \underbrace{V^\top}_{\mathbb{R}^{r\times d}}. \tag{1.33}$$

Then

$$\hat{X}^\top \hat{X} = VS^2V^\top, \tag{1.34}$$

which implies that

$$\Sigma = \frac{1}{n} V S^2 V^\top. \tag{1.35}$$

Thus **the right singular vectors of $\hat{X}$ are exactly the eigenvectors of Σ**, and the eigenvalues of Σ are the squared singular values of $\hat{X}$ scaled by $\frac{1}{n}$.

Remark

This relationship provides an alternative way to compute principal components: the top K principal directions are simply the first K right singular vectors of $\hat{X}$. This method is numerically more stable, especially when $n \neq d$ or when d is large.

Note

In practice, computing a full SVD of a $d \times d$ matrix costs $O(d^3)$ time, which is prohibitive for high-dimensional data. Modern numerical linear algebra therefore relies on approximate *SVD methods that dramatically reduce the computational burden while preserving the leading singular components.*

A common approach is the truncated SVD*: instead of computing all singular values, we only approximate the top K components. Techniques such as randomized sketching, subspace iteration, and the Nyström method project the data onto a low-dimensional subspace in which SVD becomes cheap. These methods achieve a cost of roughly*

$$O(ndK) \quad \text{or} \quad O(d^2K),$$

depending on the algorithm, reducing the complexity significantly when $K \ll d$.

This is why real-world PCA implementations (e.g., `sklearn`, `numpy.linalg.svd`*, and randomized PCA algorithms) compute only the leading singular vectors rather than performing a full decomposition.*

1.1.3 t-distributed Stochastic Neighbor Embedding (t-SNE)*

t-SNE is a nonlinear dimensionality reduction method designed for visualization of high-dimensional datasets. It constructs two probability distributions:

- one on the pairwise similarities in the high-dimensional space,
- one on the pairwise similarities in the low-dimensional embedding,

and finds an embedding that makes these two distributions as close as possible.

Definition 1.3 (High-Dimensional Similarity)

For datapoints x_i, x_j in the original space, define a conditional probability

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}. \tag{1.36}$$

The bandwidth σ_i is chosen such that the perplexity *of the distribution matches a user-specified value. The symmetric joint probability is*

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}. \tag{1.37}$$

♣

Note

It simply applies an RBF kernel to these data points.

Definition 1.4 (Low-Dimensional Similarity)

For embedding points $y_i, y_j \in \mathbb{R}^2$, t-SNE uses a Student-t distribution with one degree of freedom:

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq \ell} \left(1 + \|y_k - y_\ell\|^2\right)^{-1}}. \tag{1.38}$$

The heavy-tailed distribution alleviates the "crowding problem." ♣

Theorem 1.4 (t-SNE Objective)

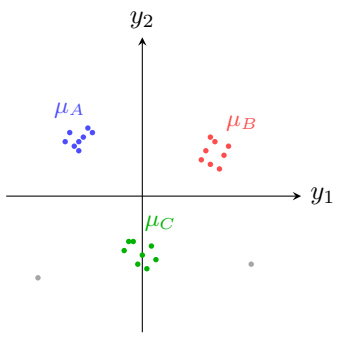
t-SNE seeks an embedding that minimizes the Kullback–Leibler divergence between the high-dimensional and low-dimensional similarity distributions:

$$\mathcal{L}(Y) = \mathrm{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}. \tag{1.39}$$

Gradient descent on $\mathcal{L}$ yields the final visualization. ♡

t-SNE preserves local neighborhood structure. Pairs with large p_{ij} are forced to satisfy large q_{ij}, keeping nearby points close. The heavy-tailed t-distribution allows distant points to be modeled far apart, preventing clusters from collapsing together.

t-SNE is excellent for visualization but not for general-purpose embedding: its geometry is not globally meaningful, and repeated runs may differ due to initialization and stochasticity.



1.2 Clustering

1.2.1 K-means Clustering Algorithm

Clustering is a fundamental task in unsupervised learning. Given a dataset $D = \{x_1, x_2, \ldots, x_n\} \subset \mathbb{R}^d$, our goal is to divide these points into K meaningful groups. Here K is a user-chosen hyperparameter, reflecting how many clusters we expect to find.

To formalize this idea, assume that each cluster $k \in [K]$ is represented by a **center** $m_k \in \mathbb{R}^d$, and each data point chooses exactly one cluster. We denote the assignment using an indicator variable $r_{ik} \in \{0, 1\}$:

$$r_{ik} = 1 \quad \Longleftrightarrow \quad x_i \text{ belongs to cluster } k. \tag{1.40}$$

Because every point must join one and only one cluster, we enforce

$$\sum_{k=1}^{K} r_{ik} = 1, \qquad \forall i \in [n]. \tag{1.41}$$

The intuitive goal of K-means is simple: each point should be close to the center of the cluster it joins. This leads to

the objective function

Theorem 1.5

$$L = \sum_{i=1}^{n} \sum_{k=1}^{K} r_{ik} \|x_i - m_k\|^2, \tag{1.42}$$

which measures the total within-cluster variance.

At first glance, minimizing L over both the assignments r and the centers m seems challenging, because the problem is non-convex and the discrete variables r_{ik} make the search space combinatorial. However, K-means becomes surprisingly tractable once we notice a key structural property of its objective. Although optimizing both the assignments r and the centers m jointly is difficult, each subproblem becomes very simple when the other variable is held fixed:

- **Fixing the centers $\{m_k\}$:** assigning each point x_i reduces to choosing the closest center.
- **Fixing the assignments $\{r_{ik}\}$:** updating each center becomes a simple averaging step, since the optimal m_k is just the mean of the points assigned to cluster k.

This observation directly motivates the alternating minimization strategy used by the K-means algorithm: we first update the assignments, then update the centers, and repeat the process until convergence. The objective value decreases at every iteration, and although the algorithm is not guaranteed to reach the global optimum, it converges to a stable **local minimum** in practice.

Assignment step (fix the centers). For each datapoint x_i, we assign it to the nearest cluster center:

$$r_{ik} = \begin{cases} 1, & k = \operatorname*{argmin}_{j \in [K]} \|x_i - m_j\|^2, \\ 0, & \text{otherwise.} \end{cases} \tag{1.43}$$

Update step (fix the assignments). To update m_k, we minimize the partial objective

$$\sum_{i=1}^{n} r_{ik} \|x_i - m_k\|^2 \tag{1.44}$$

with respect to m_k. Taking the derivative and setting it to zero,

$$\frac{\partial}{\partial m_k} \sum_{i=1}^{n} r_{ik} \|x_i - m_k\|^2 = -2 \sum_{i=1}^{n} r_{ik} (x_i - m_k) = 0, \tag{1.45}$$

which gives the closed-form mean update:

$$m_k = \frac{\sum_{i=1}^{n} r_{ik} x_i}{\sum_{i=1}^{n} r_{ik}}. \tag{1.46}$$

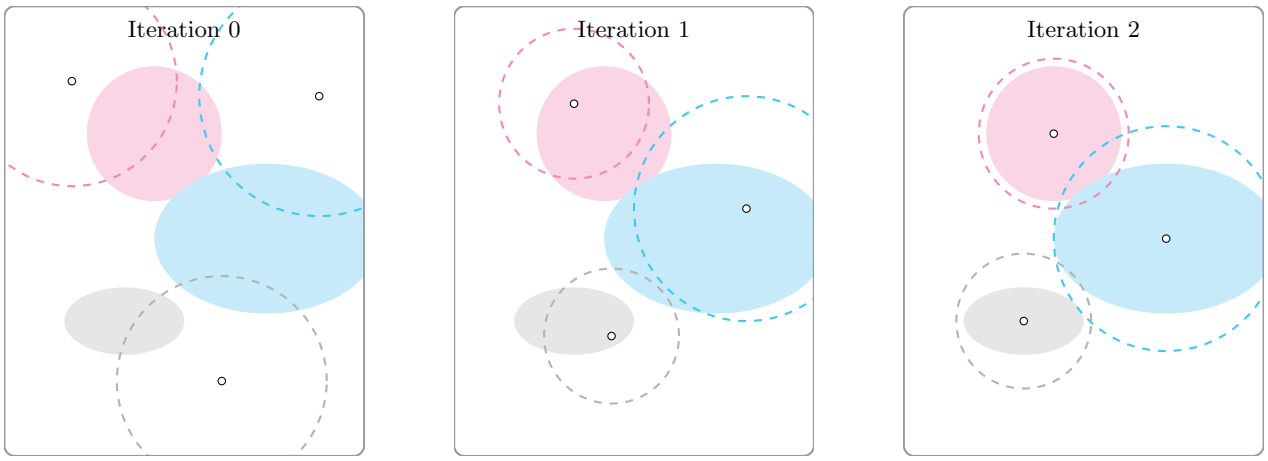
Remark

At each iteration, K-means performs a greedy improvement step: the assignment update minimizes the objective given the centers, and the center update minimizes the objective given the assignments. Hence the objective value is non-increasing and bounded below, which guarantees convergence.

Remark

However, precisely because each step is greedy and only optimizes a partial variable, K-means may converge to a local *minimum rather than the global optimum. Different initializations can therefore lead to different final solutions.*

Together, these two steps define the classical K-means iteration, which alternates between assigning points to their closest centers and recomputing each center as the average of its assigned datapoints.



Note

At each iteration, the K-means algorithm computes the centroids *of the current approximate clusters and then uses these centroids as the new cluster centers. In other words, K-means repeatedly replaces each center with the center of mass of the points currently assigned to it.*

1.2.2 Mixture of Gaussians (MoG)

K-means assumes that points belonging to each cluster are concentrated around a center, but it does not model the *shape* or *spread* of each cluster. A more expressive model is the *Mixture of Gaussians*, which assumes that points assigned to the same cluster are generated from a Gaussian distribution with its own mean and covariance.

Formally, for cluster $k \in [K]$, assume $(x \mid z_k = 1) \sim \mathcal{N}(\mu_k, \Sigma_k)$. Here $z = (z_1, \ldots, z_K)$ is a latent indicator vector with

$$z_k \in \{0, 1\}, \qquad \sum_{k=1}^{K} z_k = 1. \tag{1.47}$$

Thus $z_k = 1$ indicates that x is generated from cluster k.

The prior distribution over latent clusters is modeled using a categorical distribution:

$$P(z_k = 1) = \pi_k, \qquad 0 \le \pi_k \le 1, \qquad \sum_{k=1}^{K} \pi_k = 1. \tag{1.48}$$

Given the latent variable z, the conditional likelihood is

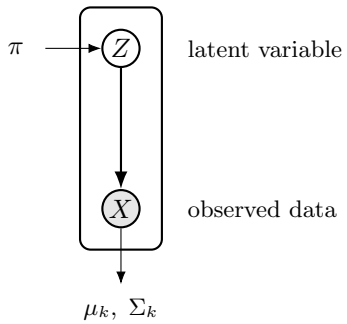
$$P(x \mid z_k = 1) = \mathcal{N}(x \mid \mu_k, \Sigma_k). \tag{1.49}$$

Since MoG is a *generative model*, the marginal density of x is obtained by summing over all latent clusters:

$$P(x) = \sum_{k=1}^{K} P(z_k = 1)\, P(x \mid z_k = 1) = \sum_{k=1}^{K} \pi_k\, \mathcal{N}(x \mid \mu_k, \Sigma_k). \tag{1.50}$$

Note

MoG can be viewed as a soft, probabilistic extension of K-means: instead of forcing each point into exactly one cluster, it allows fractional responsibilities and models not only the center but also the spread and orientation of each cluster.



We introduce the latent variable $Z := (Z_1, Z_2, \ldots, Z_K)^\top$, where each $Z_k \in \{0, 1\}$ and

$$\sum_{k=1}^{K} Z_k = 1. \tag{1.51}$$

Thus Z is a **one-hot encoding** indicating which Gaussian component generates the observation X.

Definition 1.5 (One–Hot Encoding)

For a categorical variable taking one of K possible states, its one–hot encoding *is the vector*

$$Z = (Z_1, Z_2, \ldots, Z_K)^\top \in \{0, 1\}^K, \tag{1.52}$$

where exactly one entry equals 1 *and all others equal* 0*:*

$$\sum_{k=1}^{K} Z_k = 1. \tag{1.53}$$

If the variable takes state k, then $Z_k = 1$ and $Z_j = 0$ for all $j \neq k$. ♣

Using this notation, the mixture density becomes

$$P(x) = \sum_z P(x \mid z)\, P(z) = \sum_{k=1}^{K} \pi_k\, \mathcal{N}(x \mid \mu_k, \Sigma_k). \tag{1.54}$$

We aim to estimate the parameters

$$\Theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K} \tag{1.55}$$

from data $x_1, \ldots, x_n$ via maximum likelihood estimation. The log-likelihood is

$$\operatorname*{argmax}_{\Theta} \sum_{i=1}^{n} \log P(x_i) = \operatorname*{argmax}_{\Theta} \sum_{i=1}^{n} \log\left(\sum_{k=1}^{K} \pi_k\, \mathcal{N}(x_i \mid \mu_k, \Sigma_k)\right). \tag{1.56}$$

Direct optimization is difficult because of the logarithm of a sum. In contrast, if the latent assignment were known, that is, if for each x_i we knew which $Z_{ik} = 1$, then the complete-data likelihood would factorize:

$$P(x_i, Z_{ik} = 1) = \pi_k\, \mathcal{N}(x_i \mid \mu_k, \Sigma_k), \tag{1.57}$$

and maximizing with respect to π_k, μ_k, Σ_k reduces to fitting separated Gaussian clusters, which is straightforward. This gap motivates the **Expectation–Maximization (EM) algorithm**.

EM initializes with soft assignments $\gamma_{ik} \in [0, 1]$ satisfying

$$\sum_{k=1}^{K} \gamma_{ik} = 1, \qquad \forall\, i \in [n], \tag{1.58}$$

where γ_{ik} serves as the initial responsibility of cluster k for data point x_i.

EM alternates between:

E-step: compute posterior responsibilities

$$\gamma_{ik} := P(Z_{ik} = 1 \mid x_i) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j \in [K]} \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}, \tag{1.59}$$

M-step: maximize the expected complete-data log-likelihood to update π_k, μ_k, and Σ_k:

$$\mu_k = \frac{\sum_{i \in [n]} \gamma_{ij} x_i}{\sum_{i \in [n]} \gamma_{ik}} \tag{1.60}$$

$$\Sigma_k = \frac{\sum_{i \in [n]} \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top}{\sum_{i \in [n]} \gamma_{ik}} \tag{1.61}$$

$$\pi_k = \frac{\sum_{i \in [n]} \gamma_{ik}}{n} \tag{1.62}$$